

Reasoning Shortcuts in Neuro-Symbolic Models on RAVEN’s Progressive Matrices

Juan Camacho*

June 2026

1 Introduction

Neuro-Symbolic (NeSy) models add a reasoning layer on top of a neural perception module. The neural part extracts high-level concepts $\mathbf{c}$ from a raw input $\mathbf{x}$, and the symbolic part combines those concepts with prior knowledge $\mathbf{K}$ to produce a label $\mathbf{y}$. The promise is trustworthiness: if the model gets the right answer, it should be because it has learned the right concepts and reasoned correctly over them.

Recent work has shown this promise can fail. A NeSy model can reach high accuracy on the downstream task while assigning the wrong meaning to the concepts it learns. These are called reasoning shortcuts (RSs) [1]: the model has found a different concept-to-label mapping that still satisfies $\mathbf{K}$ on the training data, even though it does not match the intended semantics.

Why do reasoning shortcuts occur? The literature points to five main causes:

- **Structure of the prior knowledge.** When $\mathbf{K}$ is symmetric or non-injective, multiple concept configurations map to the same label. A simple example is XOR: $\mathbf{K} = (c_1 \oplus c_2 \Leftrightarrow y)$. If a model swaps the meanings of c_1 and c_2 , the output is unchanged.
- **Absence of concept supervision.** Most NeSy training datasets only provide labels $\mathbf{y}$. Concepts are treated as latent variables, so nothing forces them to take their intended meaning.
- **Training-set coverage.** If the data does not contain all combinations of ground-truth concepts, the model cannot tell some configurations apart, and many alternative concept assignments remain optimal.
- **Choice of loss function.** Standard maximum-likelihood on labels only checks that the final answer is correct. As long as some concept assignment yields the right label under $\mathbf{K}$, the loss is happy. Richer losses (multitask, reconstruction, entropy) can push the model toward intended semantics.

*GitHub: <https://github.com/jucamohedano/rsbench-code/tree/raven>

- **Architectural bias of the concept extractor.** Whether the network processes objects jointly or separately changes which RSs are reachable.

The consequences of RSs include high task accuracy with corrupted concept semantics, sharp performance drops on out-of-distribution data, especially when the knowledge changes (e.g. an autonomous ambulance allowed to cross red lights in an emergency), and unreliable explanations and verification, since concept-level reasoning assumes the names match the meanings.

Motivated by these issues, **rsbench** [1] provides a benchmark suite for studying RSs across arithmetic, logical, and high-stakes tasks. In this project we extend that line of work to a different family of problems: Raven’s Progressive Matrices (RPMs), as instantiated in the RAVEN dataset [2]. RPMs are a widely used test of abstract reasoning, and their compositional structure makes them a natural candidate for NeSy modeling. Whether they are also susceptible to RSs is an open question, listed as future work in the original **rsbench** paper.

2 RAVEN’s Progressive Matrices

2.1 The task

A RAVEN problem is a 3x3 matrix of small greyscale images. The first eight cells are filled, and the ninth (bottom-right) is missing. Given an answer set of eight candidate images, the solver must pick the one that completes the matrix. The first two rows establish the rules to infer in order to solve for the third row, with the missing cell as the unknown. Figure 1 shows one example problem.

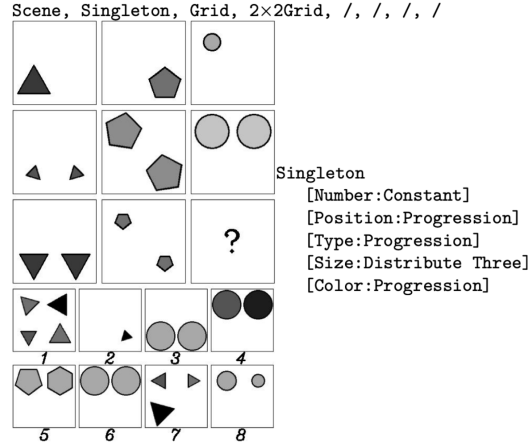


Figure 1: Example RAVEN-style matrix problem from the RAVEN’s supplementary material [2]. The top part shows the context matrix with the missing cell, and the eight candidate panels are shown below it.

In RAVEN, each cell is generated from an Attributed Stochastic Image Grammar (A-SIG), which organizes the rule-governing attributes into two levels:

- *Layout-level attributes*: NUMBER (1–9 objects per cell) and POSITION (distribution of the objects per cell). These are mutually exclusive: a non-CONSTANT rule cannot apply to both at the same time.
- *Entity-level attributes*: TYPE (shape category), SIZE (scaling factor), and COLOR (greyscale value).

Two further attributes, UNIFORMITY (layout) and ORIENTATION (entity), are noise: they vary freely and are not governed by any rule.

2.2 Rules

RAVEN implements four rule families, applied row-wise. Each rule-governing attribute is assigned exactly one rule, and all objects in the same component share the same rule set.

- **Constant.** The attribute does not change across the three panels of the row.
- **Progression.** The attribute monotonically increases or decreases along the row, with increment ± 1 or ± 2 . This gives four instantiations.
- **Arithmetic.** The third panel is the sum or the difference of the first two. ARITHMETIC is excluded on TYPE (it is not meaningful there).
- **Distribute Three.** Three values of the attribute are sampled and permuted across the rows. Each row sees all three values, in different orders.

After excluding ARITHMETIC on TYPE, RAVEN has 19 valid (rule, attribute) pairs. Each problem combines one rule per attribute, so the difficulty comes from reasoning over these independent rule assignments together.

2.3 Why RAVEN is a good candidate for studying reasoning shortcuts

RAVEN has several features that make it interesting for RS analysis:

1. **Clean perception/reasoning split.** The grammar gives a precise list of ground-truth concepts (the attribute values per cell) and a precise rule set K . This is exactly the setup `rsbench` assumes for NeSy tasks.
2. **Inherent ambiguity in the rules.** DISTRIBUTE THREE is a permutation rule: it does not care which of the three values is which, only that each row contains all three. ARITHMETIC sum is commutative ($a + b = b + a$), like MNAdd from `rsbench`.

3. **Concept supervision.** End-to-end models trained on RAVEN only see the answer index $y \in \{1, \dots, 8\}$. The attribute values are latent. This is prone to RSs, and the grammar makes it possible to check the true concept-to-label mapping.

The original RAVEN paper showed that a heuristic solver with access to the symbolic representation achieves 100% accuracy [2]. This confirms that solving RPMs reduces to a constraint satisfaction problem once perception is done correctly. In contrast, Humans achieve a mean accuracy of 84.41%, while the strongest end-to-end model achieves a 59.56%. This gap is mainly due to the lack of compositional reasoning and the one that NeSy methods are designed to close, which makes RAVEN a great fit for rsbench.

3 Methodology

3.1 A reduced RAVEN dataset: RAVEN-3x3x3

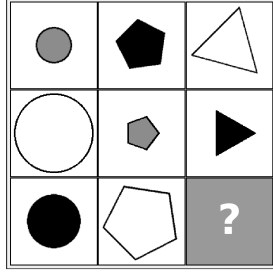
The original RAVEN attribute domains are too large to fit cleanly into a NeSy training pipeline based on Deep Probabilistic Logic (DPL). The default values are 5 shape types, 6 sizes, and 10 colors. This creates a concept space with hundreds of combinations even for a single centered object. Earlier attempts to filter the existing dataset down to a smaller domain failed. Selecting the subset of puzzles that fall within a reduced set of chosen attribute constraints retained almost no samples. The strict filtering (reducing attribute value set) and hybrid remapping of labels introduced unmapped (-1) labels.

We therefore regenerated the dataset from scratch with reduced attribute domains. Every sample is built natively over 3x3x3 values. The values are:

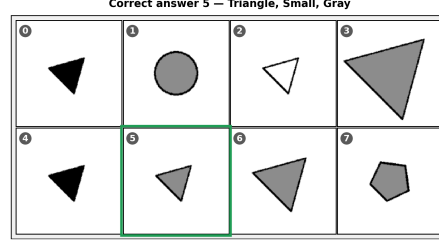
Attribute	Index 0	Index 1	Index 2
TYPE	Triangle	Pentagon	Circle
SIZE	0.4 (small)	0.6 (medium)	0.9 (large)
COLOR	255 (white)	140 (mid-grey)	0 (black)

We restricted the figure configuration to a single centered object, with NUMBER rule fixed to 1 and POSITION fixed to centered, therefore only TYPE, SIZE, and COLOR are rule-governed in practice. While for the four rule families (CONSTANT, PROGRESSION, ARITHMETIC, DISTRIBUTE THREE) we keep them unchanged.

Type: Progression | Size: Distribute_Three | Color: Distribute_Three



(a) RAVEN context matrix with the missing panel.



(b) Candidate answers with the correct solution highlighted.

Figure 2: Raven-3x3x3 example problem used in our experiments. The left panel shows the context matrix and the right panel shows the eight candidate answers.

In the generation process we used the original RAVEN codebase¹ with three modifications, all in `src/dataset/`:

1. In `const.py`: `TYPE_VALUES`, `SIZE_VALUES`, and `COLOR_VALUES` were hard-coded to the three values listed above.
2. In `main.py`: the configuration sampler was restricted to `center_single`. Two edge-case fixes were also needed because the small domains break the original sampling logic: (i) some rule combinations produce no modifiable attributes for distractor generation, so we wrapped the sampler in a retry loop; (ii) the original code needs 7 distractors but only 6 unique modifications exist with 3×2 alternatives, so we refill the pool from a backup when it empties.
3. In `Attribute.py`: the `sample_new` method (used to generate distractor values) tracks previously sampled values to avoid repeats. With only three values per attribute, this list exhausts after two calls, so we added a fallback that clears the history when empty.

We generated 5,000 problems with a 60/20/20 train/val/test split, giving 3,000 training, 1,000 validation, and 1,000 test samples. Generation took roughly 96 seconds. Verification from the built-in symbolic solver confirmed 100% accuracy on the dataset.

Following the same process, we also generate a 4x4x4 version to increase the complexity space of the task. The only difference is that we add one more value to each attribute's subset.

¹<https://github.com/WellyZhang/RAVEN>

3.2 Model and training

Our NeSy **RavenDPL** model uses DeepProbLog (DPL), a neuro-symbolic framework that integrates deep learning with probabilistic logic programming. It has two parts: a neural perception module that maps each panel to a distribution over the per-attribute concept values, and a probabilistic logic module that scores each candidate by applying the rule set $\mathbf{K}$ to those distributions. Formally, DPL separates learning and reasoning into two distinct components. The neural perception module $p_\theta(\mathbf{c} \mid \mathbf{x})$ is a standard neural network that processes a raw input $\mathbf{x}$ and outputs a probability distribution over high-level concepts $\mathbf{c}$. The probabilistic logic module $p(y \mid \mathbf{c}; \mathbf{K})$ uses symbolic prior knowledge $\mathbf{K}$ to compute the probability of an output label y given the concepts, treating each concept as an independent probabilistic fact. The full prediction is a weighted sum over all possible concept configurations:

$$p_\theta(y \mid \mathbf{x}; \mathbf{K}) = \sum_{\mathbf{c}} p(y \mid \mathbf{c}; \mathbf{K}) p_\theta(\mathbf{c} \mid \mathbf{x}),$$

where the sum ranges over all valid concept assignments. The probabilistic logic is pre-compiled into tensors, so the entire forward pass is differentiable.

Perception network. Each problem instance has 16 panels (see Fig. 2) (8 context + 8 candidates), each a 160×160 greyscale image. A shared MLP encoder, **RavenMLP**, processes all panels in parallel; its architecture is shown in Table 1.

Table 1: **RavenMLP** architecture.

INPUT	LAYER TYPE	PARAMETER	ACTIVATION
(1, 160, 160)	Flatten		
(25600)	Linear	dim=512, bias=True	ReLU
(512)	Linear	dim=128, bias=True	ReLU
(128)	Linear	dim=9, bias=True	

The 9-dimensional output is split into three blocks of three logits, one per attribute (Type, Size, Color). A softmax over each block produces the per-attribute concept distribution.

Encoding of $\mathbf{K}$. The rules are encoded as two pre-built tensors per attribute, fixed during training. We describe the 3x3x3 instance here. The reason for two tensors is that the reasoning happens at two distinct levels:

- within a single row of the matrix, and
- across the three rows

Therefore, each level of abstraction requires its own world-query tensor.

The world-queries tensor w_q handles the within-row level. It has shape $[n_{\text{vals}}^3, n_{\text{rules}}]$ and answers the question: given this attribute’s value in each of the row’s three panels, (v_1, v_2, v_3) , which rules does the row satisfy? Table 2 summarizes the supported rules for each attribute. To remove additional inherent ambiguity, we split ARITHMETIC into separate addition and subtraction rules before checking agreement across rows. Table 3 shows a representative slice of w_q for TYPE.

Table 2: Supported within-row rules by attribute. Distribute Three generalizes to $n > 3$ values by expanding into $\binom{n}{3}$ subset-specific columns; for $n = 3$, this is transparent because $\binom{3}{3} = 1$.

Attribute	n_{rules}	Supported rules
TYPE	3	Constant, Progression, Distribute Three
SIZE	5	Constant, Progression, Distribute Three, Arithmetic-Add, Arithmetic-Sub
COLOR	5	Constant, Progression, Distribute Three, Arithmetic-Add, Arithmetic-Sub

Table 3: Representative slice of the worlds-queries matrix w_q for Type. Each row is a possible world of three panel values and columns are rules. An entry is 1 if the world satisfies that rule, and 0 otherwise.

World (v_1, v_2, v_3)	Constant	Progression	Distribute Three
(Tri, Tri, Tri)	1	0	0
(Pen, Pen, Pen)	1	0	0
(Tri, Pen, Cir)	0	1	1
(Cir, Pen, Tri)	0	1	1
(Tri, Pen, Tri)	0	0	0
(Pen, Tri, Cir)	0	0	1
$\vdots$	$\vdots$	$\vdots$	$\vdots$

The AND-rule tensor handles the across-row level. It has shape $[n_{\text{rules}}^3, 2]$ and answers the question: given a candidate rule for each of the three rows, are they consistent with each other? The constraint we impose is that the same rule must apply in all three rows, so the table marks an entry as valid (column 1) only when $r_1 = r_2 = r_3$. Table 4 shows the table for Type.

For SIZE and COLOR, their respective w_q matrices have the same form with five columns (Constant, Progression, Arithmetic-Add, Arithmetic-Sub, Distribute Three), and their corresponding AND-rule table has $5^3 = 125$ rows instead of $3^3 = 27$.

Factorisation across attributes. A natural question at this point is why each attribute has its own w_q matrix instead of one global w_q matrix that

Table 4: The AND-rule table for Type. The valid column is 1 only when all three rows agree on the same rule.

(r_1, r_2, r_3)	Invalid	Valid
(Constant, Constant, Constant)	0	1
(Progression, Progression, Progression)	0	1
(Distribute Three, Distribute Three, Distribute Three)	0	1
(Constant, Constant, Progression)	1	0
(Constant, Progression, Distribute Three)	1	0
$\vdots$	$\vdots$	$\vdots$

reasons over (Type, Size, Color) jointly. The rules governing the attributes are not correlated which makes them symbolically independent and this saves compute because we don’t have to compute all of the joint possible worlds in which the rules can be arrange in one row. Without factorization, a single panel would carry $3 \times 3 \times 3 = 27$ joint states across the three attributes, and a row of three panels would have $27^3 = 19,683$ joint worlds. The worlds–queries matrix would need to define rules over these joint worlds, of the form “does this row simultaneously satisfy Type=Constant, Size=Progression and Color=Distribute Three?”. With 3 Type rules, 5 Size rules and 5 Color rules, the joint rule set has $3 \times 5 \times 5 = 75$ entries, and the global w_q would have shape $[19,683, 75]$. The factorized design replaces this with three small matrices of shape $[27, 3]$, $[27, 5]$ and $[27, 5]$. Since RAVEN’s rule grammar does not contain such rules, factorization is justified.

Computing the rule distribution for one row. Within a single row and a single attribute, the model has three panels and a categorical distribution over the three concept values for each. Treating the row panels as independent, the joint distribution over the row of values (v_1, v_2, v_3) is the outer product of the three per-panel distributions,

$$p(v_1, v_2, v_3) = p(v_1) \cdot p(v_2) \cdot p(v_3),$$

which is itself a categorical distribution over the $3^3 = 27$ worlds. Writing $w = (v_1, v_2, v_3)$ for a world and r for a candidate rule, the probability that the row is consistent with r is the total mass of the worlds that satisfy it,

$$p_{\text{row}}(r) = \sum_w p(w) \mathbb{1}[w \models r]. \quad (1)$$

The indicator $\mathbb{1}[w \models r]$ is exactly the entry $w_q[w, r]$ of the worlds–queries matrix, so this marginalization is a single matrix multiplication.

Combining rule probabilities across rows. At this step we already have the rule probability distributions $p_{\text{row}_1}(r)$, $p_{\text{row}_2}(r)$ and $p_{\text{row}_3}(r)$ for each attribute. The next reasoning step asks whether there is a single rule that explains

all three rows simultaneously. Since the third row has the last panel missing, this agreement is computed once per candidate, i.e., eight times in total. As in the within-row step, we treat the rows as independent and form the joint distribution as the outer product of the three per-row distributions. Writing $\rho = (r_1, r_2, r_3)$ for a rule triple, the joint is

$$p(\rho) = p_{\text{row}_1}(r_1) \cdot p_{\text{row}_2}(r_2) \cdot p_{\text{row}_3}(r_3),$$

which is itself a categorical distribution, now over the n_{rules}^3 possible rule triples rather than over panel-value worlds.

The role played at the within-row level by the worlds-queries matrix is played here by the AND-rule table (Table 4): it marks a rule triple ρ as valid only when $r_1 = r_2 = r_3$. Fix one rule r and ask how much probability mass sits on the triple in which all three rows follow exactly that rule. Following the same selector-and-sum pattern as Equation (1),

$$p_{\text{cand}}(r) = \sum_{\rho} p(\rho) \mathbb{1}[\rho = (r, r, r)] = p_{\text{row}_1}(r) p_{\text{row}_2}(r) p_{\text{row}_3}(r),$$

where the indicator is nonzero for only one triple, (r, r, r) , so the sum over all n_{rules}^3 triples collapses to that single term, which, by the definition of $p(\rho)$, is the product of the three rows' probabilities of following r . This is the probability that rule r governs all three rows.

For a fixed candidate, summing $p_{\text{cand}}(r)$ over the rule set gives the probability that a single rule governs all three rows on that attribute,

$$\text{score}_{\text{attr}}(\text{cand}) = \sum_r p_{\text{cand}}(r) = \sum_r p_{\text{row}_1}(r) p_{\text{row}_2}(r) p_{\text{row}_3}(r).$$

The three per-attribute scores are then summed across Type, Size and Color to give the candidate's final score.

Scoring a candidate. For each candidate $c \in \{0, \dots, 7\}$ the model computes a score using rows 1 and 2 of the matrix and the candidate plugged into the missing slot. The rule distributions of rows 1 and 2 depend only on the context panels and are computed once and shared across all candidates, i.e., for candidate c , row 3 is formed from panels $(6, 7, 8 + c)$. The three per-attribute agreement probabilities are combined to give the candidate's final score, and a softmax across the 8 candidate scores produces the predicted answer distribution $\hat{y}$. Each candidate is scored independently. The full procedure is summarized in Algorithm 1.

Loss function. The total loss is

$$\mathcal{L} = \mathcal{L}_{\text{cand}} + \gamma(w_h \mathcal{L}_{\text{entropy}} + w_c \mathcal{L}_{\text{concept}}),$$

where $\mathcal{L}_{\text{cand}}$ is negative log-likelihood on the ground-truth answer index against $\hat{y}$. $\mathcal{L}_{\text{entropy}} = 1 - H(\hat{p})$ regularizes against degenerate concept distributions, with

Algorithm 1 RavenDPL forward pass and candidate scoring.

Require: Problem instance $\mathbf{x}$ of 16 panels; shared encoder f_θ ; per-attribute worlds-queries matrices $\{w_q^{(\text{attr})}\}$ and AND-rule tables $\{\text{AND}^{(\text{attr})}\}$

- 1: $z \leftarrow f_\theta(\mathbf{x})$ $\triangleright [B, 16, 9]$ logits
- 2: Split z into three blocks and softmax each to obtain $p^{(\text{attr})} \in \mathbb{R}^{B \times 16 \times 3}$ for $\text{attr} \in \{\text{Type}, \text{Size}, \text{Color}\}$
- 3: **for** $\text{attr} \in \{\text{Type}, \text{Size}, \text{Color}\}$ **do**
- 4: **for** $i \in \{1, 2\}$ **do** $\triangleright$ Two observed rows
- 5: $p_{\text{row}_i}^{(\text{attr})} \leftarrow$ outer product of $p^{(\text{attr})}$ on the i -th row’s panels, marginalized through $w_q^{(\text{attr})}$
- 6: **end for**
- 7: **end for**
- 8: **for** $c \in \{0, \dots, 7\}$ **do**
- 9: **for** $\text{attr} \in \{\text{Type}, \text{Size}, \text{Color}\}$ **do**
- 10: $p_{\text{row}_3}^{(\text{attr})} \leftarrow$ outer product of $p^{(\text{attr})}$ on panels $(6, 7, 8 + c)$, marginalized through $w_q^{(\text{attr})}$
- 11: $s_c^{(\text{attr})} \leftarrow$ outer product of $p_{\text{row}_1}^{(\text{attr})}, p_{\text{row}_2}^{(\text{attr})}, p_{\text{row}_3}^{(\text{attr})}$, selected by the valid column of $\text{AND}^{(\text{attr})}$
- 12: **end for**
- 13: $s_c \leftarrow \sum_{\text{attr}} s_c^{(\text{attr})}$
- 14: **end for**
- 15: **return** $\hat{y} \leftarrow \text{softmax}(s_0, \dots, s_7)$

$\bar{p}$ the batch-averaged marginal per concept slot normalized by, and $\mathcal{L}_{\text{concept}}$ is per-attribute cross-entropy between the predicted concept logits and the ground-truth concept indices on the 8 context panels. Concept supervision is partial, controlled by two flags: `c_sup` sets the fraction of training samples on which it is applied, and `which_c` selects which attributes are supervised. Unsupervised entries are masked with -1 . Setting `c_sup` = 0 disables concept supervision entirely.

Log-space scoring. The scoring above is written in probability space for clarity, but in practice we compute it in log-space. In probability space the gradient starves at initialization: when the concept predictions are near-uniform, the three-way products across panels and across rows push every candidate score to almost the same value, the softmax over candidates flattens them further, and the answer-loss gradient is effectively zero. We diagnosed this by tracking the gradient norm (which collapsed from 7.7×10^{-4} to $< 10^{-7}$ within 5 steps) and the standard deviation of candidate scores (which drops to $\sim 3 \times 10^{-5}$, making the softmax over 8 candidates uniform). Computing the same quantities in log-space raises the initial gradient norm by about two orders of magnitude (from 7.7×10^{-4} to 6.8×10^{-2}). Log-space scoring on its own overshoots, collapsing concept distributions to one-hot in a single step. To avoid that we use the

entropy regularizer $\mathcal{L}_{\text{entropy}}$.

Training. We use Adam with weight decay 10^{-5} , a constant learning rate (no scheduling), and no learning rate warm-up. The entropy weight and concept supervision weight are set separately, with $\gamma = 1.0$ throughout. Hyper-parameters were selected through a learning rate sweep over $\{5 \cdot 10^{-5}, 10^{-4}, 2 \cdot 10^{-4}, 3 \cdot 10^{-4}\}$ at $w_h = 0.1$ on 2 seeds (Table 10), which identified 10^{-4} as optimal: lower rates barely escape chance and higher rates collapse to the uniform-loss plateau (full sweep in the appendix). Table 5 summarizes the three training regimes and their parameters.

Table 5: Experimental setup for the three RAVEN-3×3×3 training regimes.

Regime	w_h	c_{sup}	which_c	Seeds
No regularization	0.0	0.0	–	4
Entropy only	0.1	0.0	–	5
Entropy + sup	0.1	0.01	varied	1

All runs use $\text{lr} = 10^{-4}$, batch size = 32, 100 epochs, $w_c = 1.0$, and no early stopping. The supervision regime tests five attribute subsets `which_c` $\in \{\{0\}, \{1\}, \{2\}, \{0, 1\}, \{0, 1, 2\}\}$ on seed 123 for 40 epochs each, using concept labels on 1% of training samples (Table 7).

4 Results and Discussion

We study whether **RavenDPL** falls into reasoning shortcuts on RAVEN-3x3x3² by training it under the three regimes of increasing support shown in Table 5.

We use the concept-quality and task metrics of `rsbench`: task F1(Y) over the eight candidates, per-attribute concept F1 against the ground-truth values, and concept collapse CIs. We additionally report concept pairwise collapse and concept joint collapse. All runs use the learning rate 10^{-4} , selected by a sweep, see Table 10.

Entropy regularization. Without it, the model does not learn. Every batch, especially smaller than 128 in size, the runs freeze at the uniform loss plateau within a few epochs, and the larger-batch runs that survive reach only $\text{F1}(Y) \approx 0.23$, barely above chance (Table 11). This matches the gradient behavior described in Section 3.2: the answer signal alone is too weak to bootstrap the encoder. The entropy weight also has to be large enough, i.e., with $w_h = 0.01$ no seed escapes the plateau (Table 12), but we found that $w_h = 0.1$ is the smallest weight that reliably enables learning (Table 6).

²RAVEN-3x3x3 Weights & Biases experiments: <https://wandb.ai/jucamohedano/raven-3x3x3?nw=nwuserjucamohedano>.

A reasoning shortcut emerges. With entropy regularization ($w_h = 0.1$, no concept supervision), three of five seeds escape the plateau and reach high label F1, up to 0.91 (Table 6). The per-attribute breakdown exposes a clear reasoning shortcut: in the seeds that learn, Color reaches F1 = 1.0 while Type and Size suffer from collapse (F1 $\approx$ 0.33), shown in Figure 3. The collapse metrics show the two ungrounded attributes behave differently. Note that per-attribute collapse is zero and it’s not in the table, but pairwise collapse Cls(Size $\times$ Color) is high (0.33–0.67). The model effectively pairs each Color with a single Size, i.e., it predicts the size based on the color of the object, shown in 4), while Type is left as chance-level output because its pairwise collapse with both Size and Color is near zero for most runs. This is enough to satisfy the rules on most Raven matrices, especially for the seeds 123 and 1213. This is the pattern **rsbench** predicts and observes on Kand-Logic: high label accuracy coexisting with low concept quality, visible only once we look past F1(Y).

Table 6: Entropy only ($w_h = 0.1$, bs=32, lr= 10^{-4} , 100 epochs). In bold the rows showing clear reasoning shortcuts.

Seed	F1(Y)	F1(Type)	F1(Size)	F1(Color)	Cls(Joint)	Cls(T $\times$ S)	Cls(T $\times$ C)	Cls(S $\times$ C)
123	0.897	0.359	0.333	1.000	0.667	0.000	0.000	0.667
456	0.726	0.344	0.343	1.000	0.556	0.000	0.000	0.444
789	0.212	0.342	0.285	0.185	0.704	0.333	0.333	0.444
1011	0.338	0.336	0.348	1.000	0.556	0.000	0.000	0.333
1213	0.906	0.317	0.342	1.000	0.630	0.000	0.000	0.556
mean	0.616	0.340	0.330	0.837	0.622	0.067	0.067	0.489

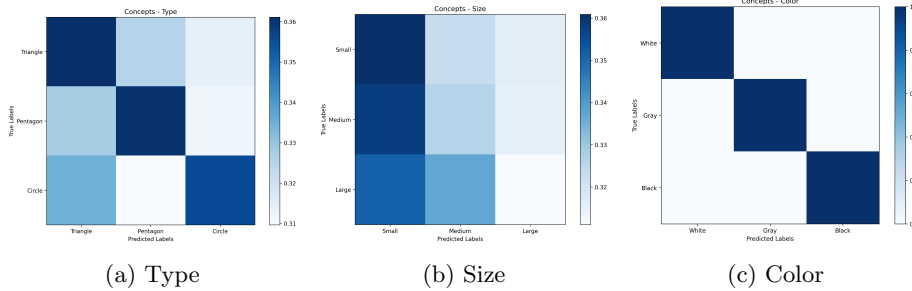


Figure 3: Per-attribute concept confusion matrices for a representative entropy-only run (seed 123; $w_h = 0.1$, no concept supervision). Each row is normalized to sum to one. Color is recovered almost perfectly, whereas Type and Size show coverage for all attribute values. Quantitative results in Table 6.

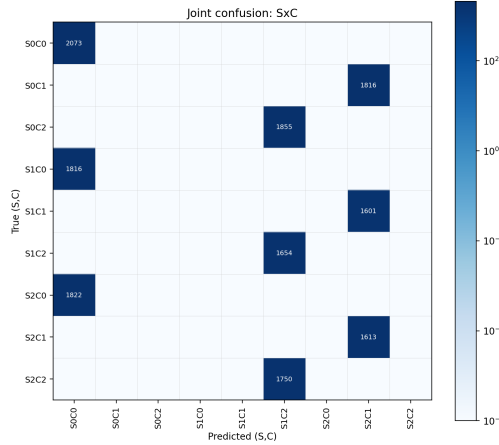


Figure 4: Size×Color pairwise concept confusion for seed run 123, over the nine (Size, Color) combinations (rows normalized). The predicted combinations concentrate on a few cells rather than spreading over all nine which is the collapse reported as $\text{Cls}(\text{Size} \times \text{Color})$ in Table 6.

Concept supervision mitigates the shortcut. Supervising the attributes breaks the shortcut, even with concept labels on only 1% of training samples (Table 7). Supervising Type and Size together raises $F1(Y)$ to 0.98, brings all three attributes to $F1 = 1.0$, and drops joint collapse to zero. Supervising a single ignored attribute is not enough: with only Type supervised, Type becomes grounded but Size stays at chance and joint collapse remains high, and the same happens, symmetrically, when only Size is supervised. Supervising Color has no effect, because the unsupervised model had already grounded Color (Table 6).

Table 7: Concept supervision as mitigation ($c_{\text{sup}} = 0.01$, $w_c = 1$, $w_h = 0.1$, seed 123, 40 epochs).

Supervised	$F1(Y)$	$F1(\text{Type})$	$F1(\text{Size})$	$F1(\text{Color})$	$\text{Cls}(\text{Joint})$	$\text{Cls}(\text{S} \times \text{C})$
Type	0.362	1.000	0.327	1.000	0.630	0.556
Type, Size	0.980	1.000	1.000	1.000	0.000	0.000
Type, Size, Color	0.983	0.991	1.000	1.000	0.000	0.000
Size	0.363	0.170	1.000	1.000	0.667	0.000
Color	0.307	0.336	0.374	1.000	0.519	0.444

Scaling to four values per attribute

To test whether the same shortcut appears in a larger concept space, we repeat the study on RAVEN-4x4x4³, where each attribute takes four values and per-attribute chance drops from 1/3 to 1/4. Learning is markedly harder here. Entropy is still mandatory, but a stronger weight ($w_h = 2.0$) and learning-rate warmup are both needed before any seed escapes the plateau, batch size of 64 over 128, and learning-rate schedules such as cosine restarts and exponential decay hurt. The full optimization sweep is summarized in Table 13. Even the best entropy-only configuration reaches only $F1(Y) = 0.54$.

The same shortcut. In the seeds that escape, the shortcut is the one we saw before: Color reaches $F1 = 1.0$ while Type and Size stay below chance, and joint collapse is near zero (Table 8). The one difference from 3x3x3 is that Size is no longer coupled with Color. Size×Color collapse is zero in the escaping seeds, so the model grounds Color and scores Type and Size low. The seeds that fail to escape (seed 789) keep high collapse and low label F1.

Table 8: Best entropy-only configuration on RAVEN-4x4x4 (entropy + 5 warmup steps, $w_h = 2.0$, bs = 64, 500 epochs). Recall that per-attribute chance score is $F1(C) = 0.25$ and per-candidate is $F(Y) = 0.125$. Per attribute concept collapse is zero and, therefore, not shown in the table.

Seed	F1(Y)	F1(Type)	F1(Size)	F1(Color)	F1(C)	Cls(Joint)	Cls(T×S)	Cls(T×C)	Cls(S×C)
123	0.510	0.237	0.217	1.000	0.484	0.109	0.000	0.000	0.000
456	0.541	0.205	0.016	1.000	0.407	0.078	0.000	0.000	0.000
789	0.273	0.260	0.237	1.000	0.499	0.641	0.000	0.000	0.438
mean	0.442	0.234	0.157	1.000	0.463	0.276	0.000	0.000	0.146

Mitigation and the ceiling. Concept supervision again removes the shortcut: with 1% of samples labeled on Type and Size, $F1(Y)$ rises to 0.96, all three attributes reach $F1 = 1.0$, and joint collapse drops to zero (Table 9). Single-attribute supervision is again insufficient. Note that supervising only Color pulls Size up to $F1 = 1.0$ as a side effect, so the clean Color signal propagates through the reasoning layer to Size, and it ties to Size. However, Type stays at chance, and $F1(Y)$ remains low (0.31). The model still needs Type and Size grounded together. The contrast between entropy-only (0.54) and concept-supervised (0.97) training locates the ceiling: the reasoning layer clearly can turn correct concepts into correct answers, so the limit is that entropy alone grounds only Color and never all three attributes at once. Pushing the encoder harder does not help: entropy annealing grounds Size and Color on one seed, but label F1 stays at 0.32 because solving a puzzle needs every governing attribute grounded and Type is still missing (Table 14).

³RAVEN-4x4x4 Weights & Biases experiments: <https://wandb.ai/jucamohedano/raven-4x4x4?nw=nwuserjucamohedano>.

Table 9: Concept supervision as mitigation on RAVEN- $4 \times 4 \times 4$ ($c_{\text{sup}} = 0.01$, $w_c = 1$, $w_h = 2.0$, seed 123, 80 epochs).

Supervised	F1(Y)	F1(Type)	F1(Size)	F1(Color)	Cls(Joint)
Type	0.279	0.969	0.447	0.253	0.656
Type + Size	0.961	1.000	1.000	1.000	0.000
Type + Size + Color	0.968	1.000	1.000	1.000	0.000
Size	0.509	0.294	1.000	0.500	0.109
Color	0.314	0.262	1.000	1.000	0.609

So, are there reasoning shortcuts? Yes, on both datasets. At three and at four values per attribute, **RavenDPL** reaches its best answer accuracy while grounding only Color and leaving Type and Size at chance. The shortcut is invisible to both $F1(Y)$ and per-attribute collapse, and surfaces only in the per-attribute concept F1 and in joint (or pairwise) collapse. This is the alternative-optimum behavior **rsbench** characterizes: the model settles on a concept-to-label mapping that satisfies the knowledge on the training data while assigning the wrong meaning to Size and Type. A small amount of concept supervision on the attributes the model would otherwise ignore removes it in both settings. Moving from three to four values per attribute changes the difficulty but not the finding. The shortcut becomes harder to escape, i.e., four values need a stronger entropy weight and a warmup phase, and fewer seeds break out of the plateau, and the best answer accuracy reachable with entropy alone drops from about 0.91 in $3 \times 3 \times 3$ to about 0.54 in $4 \times 4 \times 4$. What does not change is the shortcut itself: the same Color-only solution emerges, and the same small amount of concept supervision removes it.

A Additional 3x3x3 Results

This appendix reports the supporting experiments referenced in Section 4: the learning-rate sweep that fixes the operating point, the no-aid baseline, and the insufficient-entropy runs. The metrics are those defined in Section 4; in addition to Cls(Joint) and Cls(Size×Color) we report the remaining pairwise collapses Cls(Type×Size) and Cls(Type×Color).

Table 10: Learning rate sweep ($w_h = 0.1$, bs=32, 100 epochs, entropy). lr= 10^{-4} is optimal.

lr	Seed	F1(Y)	F1(Type)	F1(Size)	F1(Color)	Cls(Joint)
$5e-05$	123	0.169	0.259	0.330	0.000	0.889
	456	0.316	0.341	0.338	0.432	0.556
$1e-04$	123	0.897	0.359	0.333	1.000	0.667
	456	0.726	0.344	0.343	1.000	0.556
	789	0.212	0.342	0.285	0.185	0.704
	1011	0.338	0.336	0.348	1.000	0.556
	1213	0.906	0.317	0.342	1.000	0.630
$2e-04$	123	0.354	0.403	1.000	1.000	0.593
	456	0.109	0.167	0.163	0.175	0.963
$3e-04$	123	0.087	0.170	0.160	0.175	0.963
	456	0.134	0.162	0.160	0.159	0.963

Table 11: No regularization aid ($w_h = 0$, bs = 64, lr = 10^{-4} , 100 epochs). The corresponding small-batch runs (bs = 32, not shown) all froze at the uniform-loss plateau.

Seed	F1(Y)	F1(Type)	F1(Size)	F1(Color)	Cls(Joint)	Cls(T×S)	Cls(T×C)	Cls(S×C)
123	0.267	0.344	0.321	0.334	0.519	0.444	0.000	0.000
456	0.295	0.251	0.328	0.337	0.593	0.556	0.333	0.000
789	0.228	0.277	0.253	0.552	0.815	0.556	0.444	0.667
1011	0.126	0.167	0.163	0.165	0.963	0.889	0.889	0.889
mean	0.229	0.260	0.266	0.347	0.722	0.611	0.417	0.389

Concept confusion matrices

The figures below complete the confusion-matrix analysis for the representative entropy-only run (seed 123; $w_h = 0.1$, no concept supervision) whose per-attribute and Size×Color matrices appear in Section 4. In every matrix, rows are ground-truth values, columns are predicted values, and each row is normalized to sum to one.

Table 12: Insufficient entropy ($w_h = 0.01$, $bs = 64$, $lr = 10^{-4}$, 100 epochs). No seed escapes the plateau, confirming that $w_h = 0.1$ is needed for the model to bootstrap.

Seed	F1(Y)	F1(Type)	F1(Size)	F1(Color)	Cls(Joint)	Cls(T×S)	Cls(T×C)	Cls(S×C)
456	0.220	0.274	0.321	0.546	0.815	0.556	0.667	0.556
789	0.237	0.300	0.313	0.127	0.778	0.444	0.444	0.556
1011	0.080	0.167	0.163	0.175	0.963	0.889	0.889	0.889
mean	0.179	0.247	0.266	0.283	0.852	0.630	0.667	0.667

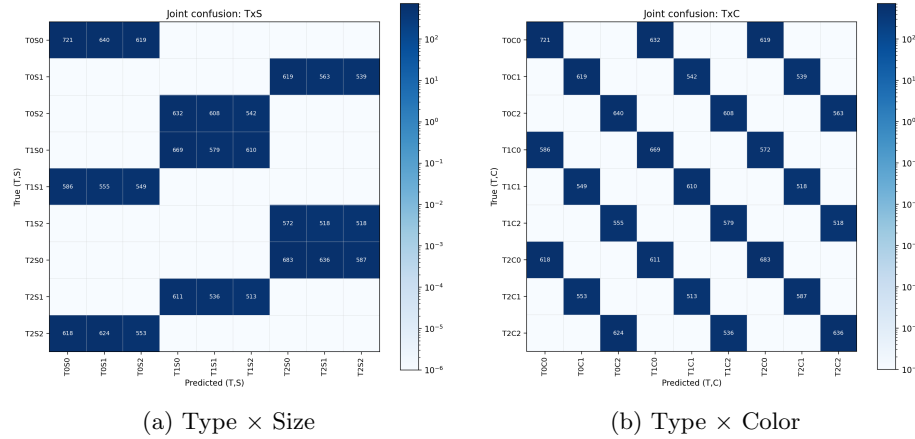


Figure 5: The remaining pairwise concept confusions for seed 123, Table 6. Predictions spread across the combinations involving Type rather than concentrating, confirming that Type is left as unstructured chance-level output (low $Cls(\text{Type} \times \text{Size})$ and $Cls(\text{Type} \times \text{Color})$), in contrast to the $\text{Size} \times \text{Color}$ collapse in Figure 4.

B Additional 4x4x4 Results

Table 13 records every training configuration tried on RAVEN-4x4x4, with the best label and concept F1 over the seeds of each. It is the source for the optimization claims in Section 4: warmup is the decisive lever, batch size 64 beats 128, and the learning-rate schedules (cosine restarts, exponential decay) and annealing do not break the plateau reached with entropy regularization.

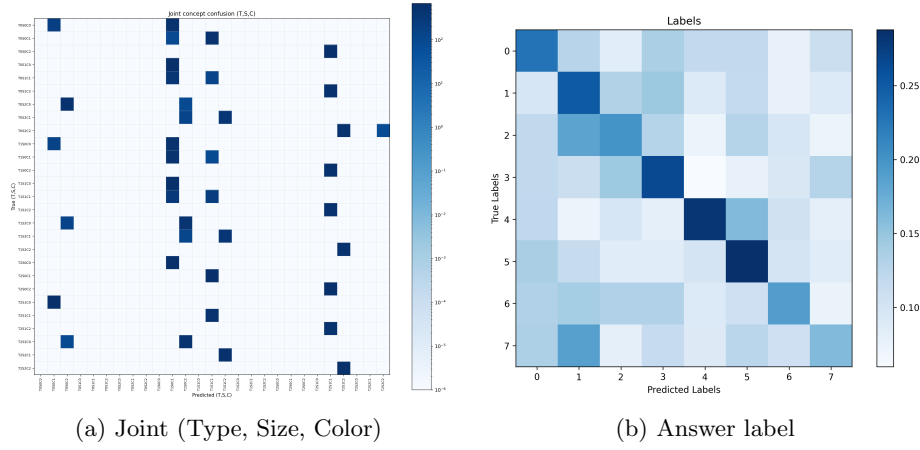


Figure 6: Joint concept confusion over the 27 (Type, Size, Color) combinations and the answer-label confusion over the eight candidates, for seed 123. The joint matrix shows the model using a reduced set of combinations (high $\text{Cls}(\text{Joint})$), while label accuracy stays high. Shows that the shortcut is invisible at the label level.

Table 13: All 4x4x4 training configurations. $\text{F1}(C)$ is the mean per-attribute concept F1; both columns report the best value over the listed seeds.

Experiment	Runs	Best $\text{F1}(Y)$	Best $\text{F1}(C)$	Seeds
No regularisation	3	0.234	0.461	3
Entropy, $w_h=2.0$, 200ep	3	0.433	0.324	3
Entropy, $w_h=2.0$, 500ep	4	0.433	0.324	4
Entropy + warmup (5ep), bs=64, 200ep	2	0.510	0.499	2
Entropy + warmup (5ep), bs=128, 200ep	2	0.299	0.496	2
Entropy + warmup (5ep), bs=64, 500ep	3	0.541	0.499	3
Entropy annealing ($w_h: 2.0 \rightarrow 0.1$)	4	0.318	0.753	4
Exponential decay ($\gamma=0.995$) + no wd	2	0.291	0.481	2
Cosine restarts ($T_0=50$)	1	0.175	0.473	1
$w_h=0.5$	2	0.196	0.322	1
$w_h=1.0$	1	0.178	0.313	1
$w_h=4.0$	2	0.264	0.318	1
Concept supervision (1% labels)	5	0.968	1.000	1

Table 14: Entropy annealing ($w_h = 2.0 \rightarrow 0.1$ over 200 epochs, bs=64, 500 epochs, warmup=5). Per-attribute collapse is zero, therefore, it is not shown in the table.

Seed	F1(Y)	F1(Type)	F1(Size)	F1(Color)	Cls(Joint)	Cls(T×S)	Cls(T×C)	Cls(S×C)
123	0.177	0.212	0.265	1.000	0.828	0.500	0.500	0.625
456	0.205	0.216	0.259	0.998	0.609	0.062	0.250	0.312
789	0.289	0.243	0.194	1.000	0.531	0.062	0.000	0.438
1011	0.318	0.263	0.996	1.000	0.672	0.562	0.000	0.000
mean	0.247	0.233	0.429	1.000	0.660	0.297	0.188	0.344

References

- [1] Samuele Bortolotti, Emanuele Marconato, Tommaso Carraro, Paolo Moret-
tin, Emile Van Krieken, Antonio Vergari, Stefano Teso, and Andrea
Passerini. A neuro-symbolic benchmark suite for concept quality and
reasoning shortcuts. *Advances in neural information processing systems*,
37:115861–115905, 2024.
- [2] Chi Zhang, Feng Gao, Baoxiong Jia, Yixin Zhu, and Song-Chun Zhu. Raven:
A dataset for relational and analogical visual reasoning. In *Proceedings of
the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*,
2019.